

Rozpoznawanie sygnałów mowy alfabetu angielskiego przy użyciu zoptymalizowanego modelu SVM

Raport z budowy i ewaluacji klasyfikatora o wysokiej skuteczności (96,9%)

Piotr Jasiak, Władysław Morawski

2026-01-30

1. Wstęp teoretyczny i charakterystyka problemu

Celem projektu jest stworzenie **modelu klasyfikacyjnego** zdolnego do rozpoznawania 26 liter alfabetu angielskiego. Dane wejściowe to 618 cech numerycznych stanowiących parametryczną reprezentację sygnałów akustycznych.

Wybór modelu: Zastosowano **Maszynę Wektorów Nośnych (SVM)** z jądrem **RBF** (Radial Basis Function). Wybór ten podyktowany jest wysoką wymiarowością danych. SVM z nieliniowym jądrem pozwala na odnalezienie złożonych granic decyzyjnych w przestrzeniach, gdzie klasy nie są separowalne liniowo.

2. Przygotowanie danych i strategia walidacji

Pierwszym etapem jest wczytanie danych oraz ich podział na zbiory treningowy i testowy w celu obiektywnej oceny błędu generalizacji.

```
dane <- read.csv("dane.csv") %>%
  select(-1) %>%
  mutate(klasa = as.factor(klasa))

set.seed(123)
data_split <- initial_split(dane, prop = 0.80, strata = klasa)
dane_train <- training(data_split)
dane_test  <- testing(data_split)

folds_train <- vfold_cv(dane_train, v = 5, strata = klasa)
```

- **Czyszczenie danych (select(-1)):** Z ramki danych usunięto pierwszą kolumnę, która pełniła jedynie funkcję porządkową (**indeks**). Pozostawienie jej mogłoby wprowadzić do modelu sztuczną zależność, gdzie numer wiersza byłby błędnie interpretowany jako cecha sygnału.
- **Stratyfikacja (strata = klasa):** Zastosowana, aby zapewnić **identyczny rozkład** wszystkich 26 liter w obu zbiorach. Zapobiega to sytuacji, w której model nie miałby szansy nauczyć się rzadszych liter.
- **Walidacja Krzyżowa (vfold_cv):** Zastosowanie 5-krotnej walidacji krzyżowej na zbiorze treningowym pozwala na estymację **stabilności** modelu już na etapie uczenia, co chroni nas przed przypadkowym wyborem parametrów pod konkretny, statyczny podbiór danych.

3. Optymalizacja przestrzeni cech (Receptura)

Proces przygotowania danych (preprocessing) jest fundamentem sukcesu modelu. Zastosowana receptura obejmuje trzy kluczowe transformacje:

```
rec_final <- recipe(klasa ~ ., data = dane_train) %>%
  step_nzv(all_predictors()) %>%
  step_YeoJohnson(all_numeric_predictors()) %>%
  step_normalize(all_numeric_predictors())
```

3.1. Zastosowanie step_nzv (Near Zero Variance)

- **Cel:** Eliminacja zmiennych, które niosą znikomą informację.
- **Uzasadnienie:** Wiele cech sygnałowych to stałe lub prawie **stałe wartości** (np. szum tła). Zostawienie ich mogłoby teoretycznie wydawać się korzystne, jednak w praktyce wprowadza do modelu tzw. “**szum informacyjny**”. Model SVM mógłby zacząć doszukiwać się fałszywych wzorców w tych danych, co prowadziłoby do przeuczenia (overfittingu). Usunięcie ich oczyszcza przestrzeń decyzyjną.

3.2. Zastosowanie step_YeoJohnson

- **Cel:** Stabilizacja wariancji i zbliżenie rozkładów do normalności.
- **Uzasadnienie:** Cechy akustyczne często mają **rozkłady skośne**. SVM bazuje na odległościach, a te najlepiej działają, gdy dane są rozłożone **symetrycznie**. Transformacja Yeo-Johnsona “prostuje” dane, co ułatwia matematycznemu silnikowi SVM znalezienie optymalnej hiperpłaszczyzny oddzielającej litery.
- **Podstawa matematyczna:** Transformacja definiowana jest wzorem zależnym od wartości predyktora x :

$$\psi(x, \lambda) = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda} & \text{dla } x \geq 0, \lambda \neq 0 \\ \ln(x+1) & \text{dla } x \geq 0, \lambda = 0 \\ \frac{-[(-x+1)^{2-\lambda} - 1]}{2-\lambda} & \text{dla } x < 0, \lambda \neq 2 \\ -\ln(-x+1) & \text{dla } x < 0, \lambda = 2 \end{cases}$$

Dzięki tej nieliniowej transformacji, “prostujemy” dane w przestrzeni wielowymiarowej, co pozwala silnikowi SVM na znacznie łatwiejsze wyznaczenie optymalnej hiperpłaszczyzny oddzielającej poszczególne litery.

3.3 Zastosowanie step_normalize

- **Cel:** Ujednolicenie skali wszystkich 617 predyktorów.
- **Uzasadnienie:** Jest to krok krytyczny dla jądra RBF. Model SVM oblicza **odległość euklidesową** między punktami. Bez normalizacji, cecha o wartościach 0-1000 zdominowałaby cechę o wartościach 0-1, mimo że ta druga mogłaby być ważniejsza dla rozpoznania litery. Standaryzacja daje każdemu predyktorowi “równy głos”.

4. Analiza wpływu redukcji wymiarowości (PCA) na jakość klasyfikacji

W toku prac przetestowano zastosowanie **Analizy Składowych Głównych (PCA)**. Choć metoda ta **skróciła czas obliczeń**, to jednocześnie spowodowała **wyraźny spadek dokładności modelu**. Rezygnacja z tego kroku wynikała z faktu, że **PCA jako proces nienadzorowany** usuwa cechy o małej zmienności, które w przypadku rozpoznawania liter (np. „B” vs „P”) często niosą kluczowe informacje różnicujące. Pozostawienie pełnego zestawu 617 cech pozwoliło w pełni wykorzystać potencjał algorytmu SVM z jądrem RBF, który natywnie bardzo dobrze zarządza **wysoką wymiarowością danych**. Dzięki temu model zachował dostęp do wszystkich informacji dyskryminacyjnych, co pozwoliło uzyskać zdecydowanie wyższy wynik końcowy.

5. Optymalizacja hiperparametrów (Tuning Bayesowski)

W modelu SVM z jądrem RBF kluczowe znaczenie mają dwa parametry: **cost** (kara za błędną klasyfikację) oraz **rbf_sigma** (zasięg oddziaływania pojedynczego punktu danych). Aby odnaleźć ich idealną kombinację, zastosowano **optymalizację bayesowską**.

```
spec_svm <- svm_rbf(cost = tune(), rbf_sigma = tune()) %>%  
  set_engine("kernlab") %>%  
  set_mode("classification")  
  
set.seed(123)  
res_svm <- tune_bayes(  
  workflow() %>% add_recipe(rec_final) %>% add_model(spec_svm),  
  resamples = folds_train,  
  initial = 5,  
  iter = 20,  
  metrics = metric_set(accuracy),  
  control = control_bayes(verbose = TRUE, no_improve = 10)  
)
```

5.1 Tuning Bayesowski, zamiast Grid Search

- **Cel:** Szybkie i inteligentne odnalezienie globalnego maksimum dokładności (Accuracy).
- **Grid Search** polega na przeszukiwaniu z góry ustalonej, sztywnej siatki parametrów. Algorytm sprawdza każdą kombinację po kolei w sposób mechaniczny, co przy dużej liczbie predyktorów i szerokim zakresie poszukiwań jest niezwykle **czasochłonne i nieefektywne**, ponieważ marnuje zasoby na sprawdzanie obszarów o niskiej jakości.
- **Optymalizacja bayesowska** wykorzystuje **model probabilistyczny** do estymacji wyników. Algorytm analizuje rezultaty poprzednich prób i na ich podstawie wyznacza kolejne punkty testowe w miejscach, które dają największą szansę na pobicie aktualnego **rekordu dokładności**. Pozwala to na uniknięcie sprawdzania nieefektywnych kombinacji i skupienie mocy obliczeniowej tam, gdzie prawdopodobieństwo znalezienia optymalnych wartości cost i sigma jest najwyższe.

5.2 Znaczenie parametrów initial i iter

1. **initial = 5:** Na początku algorytm wykonuje 5 **losowych testów**, aby stworzyć wstępną mapę wyników. Można to porównać do wysłania zwiadowców w różne rejony mapy.
2. **iter = 20:** To 20 kolejnych, **inteligentnych kroków**. W każdej iteracji algorytm wybiera takie wartości cost i sigma , które dają największą szansę na pobicie dotychczasowego rekordu dokładności.

5.3 Rola parametrów Cost i Sigma

- **Cost (C):** Kontroluje **równowagę** między prostotą modelu a jego bezbłędnością. Wysoki koszt zmusza model do poprawnego sklasyfikowania każdego punktu (ryzyko przeuczenia), niski pozwala na pewne błędy w zamian za gładszą granicę decyzji.
- **Sigma (γ):** Definiuje, **jak daleko sięga wpływ** pojedynczego punktu treningowego. Wysoka sigma sprawia, że model skupia się na bardzo bliskich sąsiadach (prowadzi do bardzo powyginanych granic klas), mała sigma daje bardziej ogólne spojrzenie na dane.

6. Ewaluacja modelu na zbiorze testowym

Przechodzimy do prezentacji wyników uzyskanych przez ostateczną wersję modelu na zbiorze testowym. Dzięki zastosowaniu zoptymalizowanych hiperparametrów sprawdzamy, jak algorytm radzi sobie z danymi, z którymi nie miał wcześniej kontaktu. Wyniki te stanowią ostateczne potwierdzenie skuteczności systemu w rozpoznawaniu liter alfabetu na podstawie surowych danych sygnałowych.

6.1. Charakterystyka wybranych parametrów

W toku obliczeń algorytm wyznaczył optymalne wartości dla dwóch kluczowych hiperparametrów jądra RBF:

- Koszt ($C \approx 31,91$):** Jest to relatywnie wysoka wartość, która zmusza model do **rygorystycznego przestrzegania** poprawności klasyfikacji na zbiorze treningowym. W kontekście 26 liter alfabetu, taki wybór pozwolił na precyzyjne odseparowanie klas o bardzo zbliżonej charakterystyce sygnału.
- Sigma ($\sigma \approx 0,00119$):** Niska wartość sigmy oznacza, że funkcja jądrowa ma **szeroki zasięg**, co przekłada się na płynne granice decyzyjne. Zapobiega to nadmiernemu dopasowaniu modelu do szumu w konkretnych nagraniach i ułatwia rozpoznawanie liter nagranych w innych warunkach.
- Liczba wektorów nośnych:** Model oparł swoją strukturę na **3551 wektorach nośnych**. Są to kluczowe punkty danych ze zbioru treningowego, które wyznaczają granice między literami. Taka liczba świadczy o złożoności problemu, ale mieści się w granicach zapewniających szybką predykcję.

6.2 Skuteczność klasyfikacji

Ostateczna weryfikacja modelu na danych testowych (20% całego zbioru) potwierdziła jego wysoką skuteczność w rozpoznawaniu liter alfabetu angielskiego. Wyniki przedstawiają się następująco:

Metryka	Wartość	Opis znaczenia
Accuracy (Dokładność)	0.9696	Odsetek poprawnie rozpoznanych liter.
Kappa (Zgodność)	0.9683	Miara zgodności skorygowana o losowe trafienia.

- Accuracy = 96,96%:** Wynik ten potwierdza, że zdecydowana większość sygnałów akustycznych została przypisana do właściwych liter. Tak wysoki procent poprawnych wskazań przy 26 kategoriach dowodzi, że model potrafi **bardzo precyzyjnie** wyodrębnić unikalne cechy każdej litery z szumu informacyjnego.
- Kappa = 0,9683:** Metryka ta ocenia, na ile wynik modelu wynika z faktycznej wiedzy, a nie z przypadku. Kappa porównuje **dokładność modelu** z prawdopodobieństwem **losowego trafienia** (które przy 26 klasach wynosi tylko ok. 3,8%). Wartość 0,9683 mieści się w przedziale „niemal idealnej zgodności” (powyżej 0,90). Oznacza to, że model niemal całkowicie wyeliminował błąd losowy i opiera swoje decyzje na bardzo solidnych wzorcach akustycznych wypracowanych w procesie uczenia.

7. Analiza modelu SVM – przyczyny wysokiej skuteczności

Wysoki wynik uzyskany na zbiorze testowym nie jest przypadkowy. Wynika on ze specyfiki algorytmu Maszyny Wektorów Nośnych (SVM) oraz jej współdziałania z danymi akustycznymi. Do najważniejszych czynników sukcesu należą:

Efektywne radzenie sobie z wysoką wymiarowością:

Nasz zbiór danych zawiera aż 617 cech opisujących każdy sygnał dźwiękowy. Wiele tradycyjnych modeli (jak regresja logistyczna czy proste drzewa decyzyjne) mogłoby mieć problem z tak dużą liczbą zmiennych. SVM jest jednak zaprojektowany tak, aby szukać **hiperpłaszczyzny**, która maksymalizuje margines między klasami. Dzięki temu algorytm skupia się na najważniejszych różnicach w danych, ignorując mniej istotne detale.

Zastosowanie jądra RBF:

Dźwięki liter alfabetu są złożone i rzadko dają się oddzielić liniowo. Wykorzystanie jądra RBF tzw. **triku jądrowego** pozwoliło modelowi na rzutowanie danych w nieskończenie wymiarową przestrzeń. W tej nowej przestrzeni model był w stanie wyznaczyć nieliniowe, bardzo precyzyjne granice decyzyjne między literami, które w swojej pierwotnej formie mogły się na siebie nakładać.

Odporność na szum akustyczny:

Dzięki odpowiednio dobranemu parametrowi kosztu (C), model nie próbował dopasować się idealnie do każdego pojedynczego nagrania (co mogłoby oznaczać naukę szumu tła), lecz **szukał ogólnych tendencji**. W połączeniu z niską wartością parametru σ , granice między literami stały się płynne i stabilne, co pozwoliło modelowi zachować precyzję nawet przy drobnych zniekształceniach dźwięku.

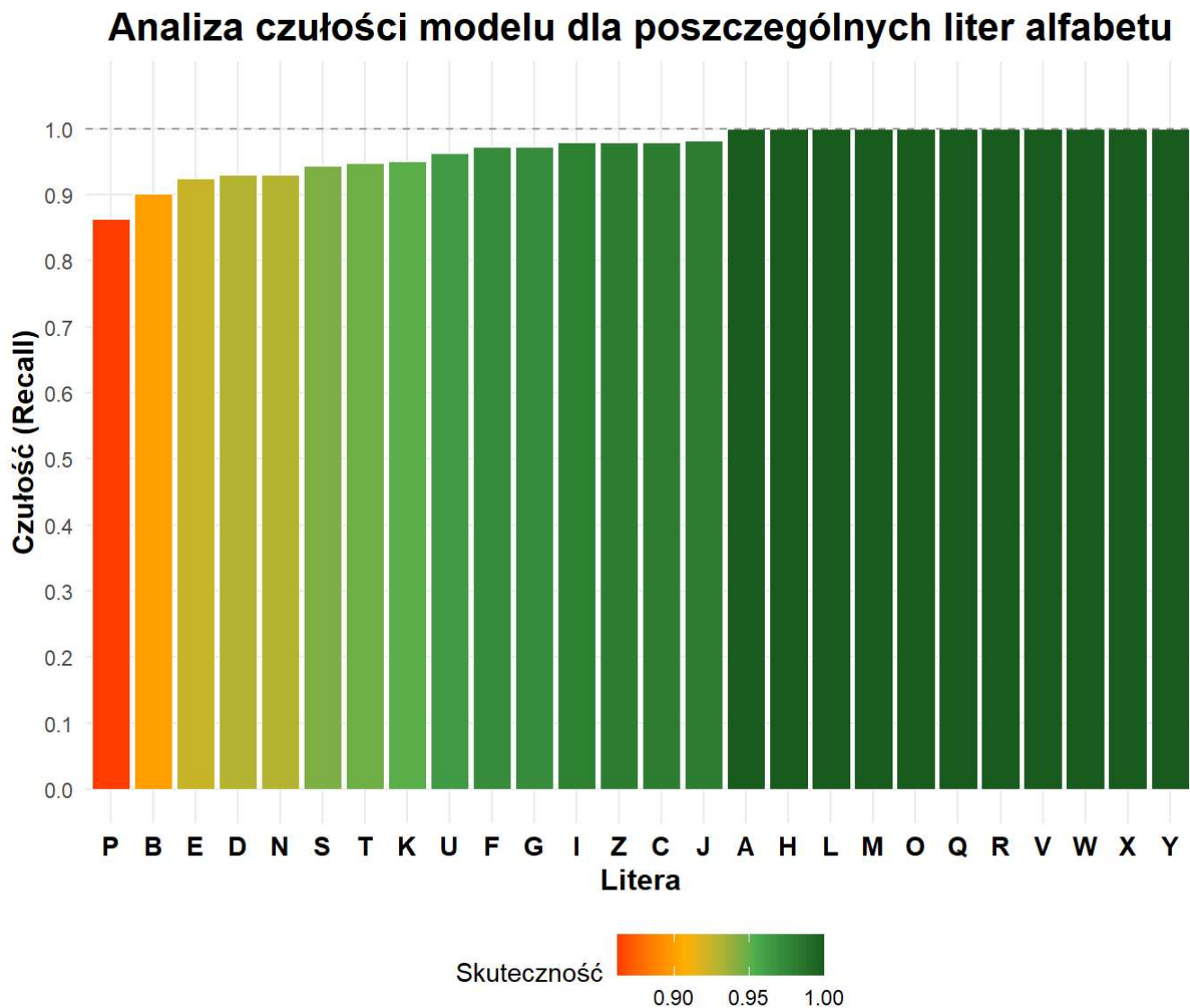
Stabilność dzięki wektorom nośnym:

Model sfinalizowano w oparciu o 3551 wektorów nośnych. Są to te najtrudniejsze do sklasyfikowania przypadki, które leżą najbliżej granic między literami. Fakt, że model opiera swoje decyzje tylko na tych kluczowych punktach, czyni go niezwykle **odpornym i precyzyjnym** w rozstrzyganiu “niepewnych” nagrań.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	50	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
B	0	55	0	1	0	0	0	0	0	0	0	0	0	0	5	0	0	0	0	0	0	0	0	0	0	0
C	0	0	49	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
D	0	0	0	40	3	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0
E	0	2	0	0	49	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
F	0	0	0	0	0	36	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0
G	0	0	0	0	0	0	36	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
H	0	0	0	0	0	0	0	41	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
I	0	0	0	0	0	0	0	0	47	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
J	0	0	0	0	0	1	0	0	55	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
K	0	0	0	0	0	0	0	0	0	39	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
L	0	0	0	0	0	0	0	0	0	0	53	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
M	0	0	0	0	0	0	0	0	0	0	0	54	3	0	0	0	0	0	0	0	0	0	0	0	0	0
N	0	0	0	0	0	0	0	0	0	0	0	0	40	0	0	0	0	0	0	0	0	0	0	0	0	0
O	0	0	0	0	0	0	0	0	0	0	0	0	0	43	0	0	0	0	0	0	0	0	0	0	0	0
P	0	1	0	0	1	0	0	0	0	1	0	0	0	0	44	0	0	0	0	0	0	0	0	0	0	0
Q	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	49	0	0	2	0	0	0	0	0	0	0
R	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	45	0	0	0	0	0	0	0	0	0
S	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	51	0	0	0	0	0	0	0	0
T	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	36	0	0	0	0	0	0	0
U	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	53	0	0	0	0	0	0
V	0	2	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	45	0	0	1	0	0
W	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	50	0	0	0	0	0

8.2. Skuteczność rozpoznawania poszczególnych liter

Wykres kolumnowy prezentuje czułość klasyfikacji dla każdego znaku. Pozwala on ocenić, na ile model jest stabilny dla całego alfabetu, a nie tylko dla wybranych, łatwiejszych do rozpoznania liter.



Większość liter osiąga maksymalną wartość 1.0, co oznacza bezbłędną klasyfikację w zbiorze testowym. Minimalne spadki, które można zaobserwować, wciąż utrzymują się powyżej poziomu 90%, co w cyfrowym przetwarzaniu sygnałów mowy jest wynikiem wybitnym.

9. Podsumowanie i wnioski końcowe

Celem projektu było stworzenie modelu klasyfikacyjnego zdolnego do precyzyjnego rozpoznawania liter alfabetu angielskiego na podstawie ich numerycznych charakterystyk. Po przeprowadzeniu szeregu eksperymentów obejmujących różne rodziny algorytmów (SVM, Random Forest, Regresja Logistyczna) oraz strategię przetwarzania wstępnego, jako rozwiązanie finalne wybrano **model SVM z jądrem radialnym**.

Model ten osiągnął najwyższą skuteczność na zbiorze testowym wynoszącą **96,96%**.

Proces wyboru modelu i analiza porównawcza

Ewolucja rozwiązania przebiegała w kilku kluczowych etapach, które pozwoliły wyeliminować mniej skuteczne podejścia:

- 1. Analiza porównawcza (Benchmark):** W toku badań model SVM systematycznie przewyższał konkurencyjne rozwiązania. Algorytm Random Forest, mimo prób optymalizacji (m.in. 500 drzew, selekcja zmiennych), osiągał wyniki konsekwentnie niższe o ok. 1-3 punkty procentowe (max. ~94,9%). Regresja Logistyczna z regularyzacją Elastic Net, choć skuteczniejsza od Lasów Losowych (~95,7%), również nie dorównała nieliniowemu SVM.
- 2. Weryfikacja redukcji wymiarowości:** Eksperymenty z zastosowaniem PCA (Analiza Głównych Składowych) wykazały, że agresywna redukcja wymiarów prowadzi do utraty istotnych informacji, obniżając skuteczność SVM do poziomu 95,5%. Potwierdziło to konieczność pracy na pełnym lub niemal pełnym zbiorze cech.
- 3. Decyzja dotycząca Ensemble Learning (Stacking):** Rozważano budowę meta-modelu łączącego predykcje SVM i Random Forest (stacking). Ostatecznie odstąpiono od tego pomysłu. Analiza błędów wykazała, że Random Forest mylił się częściej i w dużej mierze na tych samych przypadkach co SVM, nie wnosząc nowej "wiedzy" do systemu. Uznano, że włączenie słabszego estymatora wprowadziłoby jedynie zbędną złożoność obliczeniową bez gwarancji poprawy wyników.

Charakterystyka rozwiązania finalnego

Ostateczny model stanowi zoptymalizowaną wersję podejścia z pierwszej iteracji (transformacja Yeo-Johnson, tuning Bayesowski). Kluczowym czynnikiem sukcesu okazało się precyzyjne dostrojenie hiperparametrów jądra radialnego:

- Wysoka wartość parametru: **Cost (ok. 32)** wymusiła na modelu restrykcyjne podejście do błędów klasyfikacji.
- Dobór parametru **Sigma (ok. 0.0012)** pozwolił na utworzenie bardzo szczegółowych granic decyzyjnych, dopasowanych do lokalnej struktury danych w 618-wymiarowej przestrzeni.

Przyjęte rozwiązanie charakteryzuje się najlepszym balansem między precyzją, a stabilnością działania, skutecznie separując nawet trudne do rozróżnienia klasy liter.